Building data course — Week 5 — Haystack RDF and external timeseries references

sparql-playground / lessons

Generated for print and offline reading

Week 5 — Haystack RDF and external timeseries references

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. Each lesson's two scripts are printed after it. Keep notes under the ignored `student-work/` folder.

Contents

- Day 21 — Why hard-coded names break: one AHU's `mad_c` is the other's `mad_c_pct`
- Day 22 — Haystack tags as RDF: find a rule's inputs by tags with SPARQL
- Day 23 — UUIDs and the long table: stable ids, `samples`, and pivoting long to wide
- Day 24 — External timeseries references: one model, history in SQLite or in a CSV
- Day 25 — Model-driven FC1 through the app's API (week 5 challenge)

Day 21 — Why hard-coded names break: one AHU's `mad_c` is the other's `mad_c_pct`

Goal: see a rule that works on one AHU fail on the next, then fix it with the smallest possible model: a map from *role* ("outside-air damper") to *column* ("mad_c" or "mad_c_pct").

Read

Weeks 1–4 typed column names into every script: `supply_fan_status`, `da_p_inwc`, `mad_c`. That was honest and it worked, because we picked columns that happen to exist under the same name on both AHUs. Not every column does. AHU_1's outside-air damper command is `mad_c`. AHU_2's is `mad_c_pct`. Same physical thing, same unit (percent open), different name in the BAS export. Ask both AHUs for `mad_c` and one of them fails:

	`day21_pandas.py`	`day21_sql.py`
ask for `mad_c` on AHU_2	`pd.read_csv(..., usecols=[..., "mad_c"])` raises `ValueError`	`SELECT mad_c FROM ahu_2` raises `no such column`

You can patch this with `if equip == "AHU_2": col = "mad_c_pct"`. Do that for 30 columns and 40 AHUs and the `if`s *are* the data model, written in code and copied into every script. A change of building means editing every script.

The way out is to keep the facts **out of the rule**. The rule says "I need the outside-air damper and the fan status". Something else, a **model**, says which column holds each. The smallest model is already in the data folder: `data/BUILDING_50/AHU_*/column_map.json`, a dictionary from role to column:

```
"outside-air-damper": "mad_c"          // AHU_1
"outside-air-damper": "mad_c_pct"      // AHU_2
```

Open-FDD calls the left-hand side a **role** (`fan-status`, `duct-static-pressure`, …). Now the script loops over AHUs, looks up the column for each role, and reads *that*. One script, any number of AHUs, and the only thing that changes between buildings is the map.

How each half uses the map:

Step	`day21_pandas.py`	`day21_sql.py`
role → column	`col_map[equip][role]`	the same dict, from the same JSON
load by role	`read_csv(usecols=[...]).rename(columns={col: shared_name})`	`SELECT "mad_c_pct" AS oa_damper_pct FROM "ahu_2"`
stack AHUs	`pd.concat`	`UNION ALL`
summary	`groupby("equipment_id").agg(...)`	`GROUP BY equipment_id`

Renaming vs building SQL. pandas just renames the loaded columns to shared names, so everything after the load is identical for both AHUs. SQL has no "rename after the fact": the query text itself must differ per AHU, so the script *builds* it from the map. That brings a rule you should never forget: a **value** can be a bound parameter (`:eq0`), but an **identifier** (a table or column name) cannot. `SELECT :col FROM t` returns the string, not the column. So identifiers get pasted into the SQL text, and text pasted into SQL is how SQL injection happens. The defence is a **whitelist**: accept a name only if it is a real column of the real table (`PRAGMA table_info`), then quote it (`"`, doubled if it appears inside). The script's `quote_ident` does both. Today the map is a file you trust, but the habit costs nothing and matters the first time a name comes from a user or a web form.

Run

```
uv run python lessons/course.py run 21
```

That runs `day21_pandas.py`, then `day21_sql.py`. It reads only the CSVs and `var/lessons.sqlite`. Both print the failed hard-coded attempt, the damper column per AHU and the summary:

```
Hard-coded `mad_c` on each AHU:        AHU_1 worked, AHU_2
did not
Damper column per AHU, from column_map.json:
equipment_id damper_column
       AHU_1         mad_c
       AHU_2     mad_c_pct
Outside-air damper while the supply fan runs (one script,
both AHUs):
equipment_id damper_column  run_rows  run_hours
mean_damper_pct  open_over_50_h  closed_h
       AHU_1         mad_c     18893    1574.42
42.88          605.83    357.83
       AHU_2     mad_c_pct     17995    1499.58
51.90          696.42    347.75
```

(The error *message* differs between pandas and SQLite; only whether it worked is compared.) While the fan runs, AHU_1's damper averages 43% open and AHU_2's 52%. Both spend about 350 running hours fully closed.

Try it

1. Add the mixed-air temperature (role `mixed-air-temp`) to both scripts and report its mean while the fan runs. Which lines change? Which don't?
2. In the SQL script, change one role's column in the map to `"mad_c; DROP TABLE ahu_1"` (just in the Python dict). What happens, and why is that good?

Answers

1. Add `"mixed-air-temp": "mat"` to `ROLES` (the shared name is yours to choose) and one more aggregate in the summary. The loading code doesn't change: it loops over `ROLES`. In the SQL script the generated `SELECT` gains one more column automatically. Adding a role is data, not control flow.
2. `quote_ident` raises `ValueError: unknown column ...` before any SQL runs, because that string is not a column of the

table. Without the whitelist, the string would have been pasted into the query text.

Why it matters

A column map is a model: it separates *what the rule needs* from *where this building keeps it*. But it is one JSON file per AHU, it only knows columns in one CSV, and it can't say that the supply fan is *part of* AHU_1 or that a sensor reads in °F. Tomorrow you meet a richer model: Haystack tags written as RDF.

day21_pandas.py

```
"""Day 21, pandas: an outside-air damper summary across
         AHUs whose column names differ.

    uv run python lessons/unit3-modeling/day21_pandas.py

AHU_1's damper command is the column `mad_c`; AHU_2's is
         `mad_c_pct`. A script with the name typed
in works for one AHU and fails on the other.
         column_map.json (role -> column, one per AHU) is
         the
first, smallest "model": the script asks for a *role* and
         the map says which column holds it.
Compare the printout with day21_sql.py: the numbers should
         match."""

import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
EQUIPMENT = ("AHU_1", "AHU_2")

# 1. The hard-coded way: ask for `mad_c` on every AHU. It
         works for AHU_1 and fails for AHU_2.
HARD_CODED = ["timestamp_utc", "supply_fan_status",
         "mad_c"]
tried = []
for equip in EQUIPMENT:
    try:
        pd.read_csv(DATA / equip / "history_wide.csv",
         usecols=HARD_CODED, nrows=5)
        tried.append((equip, True, "ok"))
    except ValueError as err:
        tried.append((equip, False, str(err)))
hardcoded = pd.DataFrame(tried, columns=["equipment_id",
         "worked", "message"])

# 2. The first model: column_map.json says which column
         holds each role, per AHU.
ROLES = {"fan-status": "fan_status", "outside-air-damper":
         "oa_damper_pct"}  # role -> our own name
col_map = {e: json.loads((DATA / e /
         "column_map.json").read_text())["points"] for e
         in EQUIPMENT}
damper_columns = pd.DataFrame(
```

```
    [(e, col_map[e]["outside-air-damper"]) for e in
         EQUIPMENT],
    columns=["equipment_id", "damper_column"],
)

# 3. Load by role: look up each AHU's column names, read
         only those, rename to the shared names.
frames = []
for equip in EQUIPMENT:
    by_role = {col_map[equip][role]: name for role, name
         in ROLES.items()}  # column -> shared name
    df = pd.read_csv(DATA / equip / "history_wide.csv",
         usecols=["timestamp_utc", *by_role]).rename(
        columns=by_role
    )
    df.insert(0, "equipment_id", equip)
    frames.append(df)
oa = pd.concat(frames, ignore_index=True)

# 4. One summary for both AHUs: only rows where the supply
         fan runs (the damper is moot when off)
running = oa[oa["fan_status"] == 1]
summary = (
    running.groupby("equipment_id")["oa_damper_pct"]
    .agg(run_rows="size", mean_damper_pct="mean")
    .assign(
        open_over_50_h=running.assign(hit=running["oa_damper_pct"]
         > 50)
        .groupby("equipment_id")["hit"]
        .sum()
        * 5
        / 60,
        closed_h=running.assign(hit=running["oa_damper_pct"]
         <= 0)
        .groupby("equipment_id")["hit"]
        .sum()
        * 5
        / 60,
    )
    .reset_index()
    .merge(damper_columns, on="equipment_id")
)
summary.insert(2, "run_hours", summary["run_rows"] * 5 /
         60)
summary = summary[
    [
        "equipment_id",
        "damper_column",
        "run_rows",
        "run_hours",
```

```
        "mean_damper_pct",
        "open_over_50_h",
        "closed_h",
    ]
]

RESULTS = {
    "hardcoded_worked": hardcoded[["equipment_id",
         "worked"]],
    "damper_columns": damper_columns,
    "damper_summary": summary,
}

print("Hard-coded `mad_c` on each AHU:")
print(hardcoded.to_string(index=False, max_colwidth=60))
print("\nDamper column per AHU, from column_map.json:")
print(damper_columns.to_string(index=False))
print("\nOutside-air damper while the supply fan runs (one
         script, both AHUs):")
print(summary.round(2).to_string(index=False))
```

day21_sql.py

```python
"""Day 21, SQL: an outside-air damper summary across AHUs whose column names differ.

    uv run python lessons/course.py setup          # once: loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit3-modeling/day21_sql.py

The same computation as day21_pandas.py, done by SQLite. The column names come from
column_map.json, so the SQL text is *built*. A value can be a bound parameter (:eq0), but a table
or column name cannot, so every name from the map is checked against the table's real columns (a
whitelist) and then quoted. Compare the printout with day21_pandas.py: the numbers should match."""

import json
import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python lessons/course.py setup")
conn = sqlite3.connect(DB)

EQUIPMENT = {"AHU_1": "ahu_1", "AHU_2": "ahu_2"}  # equipment id -> table (the whitelist of tables)

# 1. The hard-coded way: ask for `mad_c` on every AHU. It works for AHU_1 and fails for AHU_2.
tried = []
for equip, table in EQUIPMENT.items():
    try:
        conn.execute(f'SELECT timestamp_utc, supply_fan_status, mad_c FROM "{table}" LIMIT 5')
        tried.append((equip, 1, "ok"))
    except sqlite3.OperationalError as err:
        tried.append((equip, 0, str(err)))
hardcoded = pd.DataFrame(tried, columns=["equipment_id", "worked", "message"])
```

```
# 2. The first model: column_map.json says which column
        holds each role, per AHU.
ROLES = {"fan-status": "fan_status", "outside-air-damper":
        "oa_damper_pct"}  # role -> our own name
col_map = {e: json.loads((DATA / e /
        "column_map.json").read_text())["points"] for e
        in EQUIPMENT}
damper_columns = pd.DataFrame(
    [(e, col_map[e]["outside-air-damper"]) for e in
        EQUIPMENT],
    columns=["equipment_id", "damper_column"],
)


def quote_ident(name: str, allowed: set[str]) -> str:
    """Identifiers can't be bound as parameters: accept a
        name that really exists, then quote it."""
    if name not in allowed:
        raise ValueError(f"unknown column {name!r}")
    return '"' + name.replace('"', '""') + '"'


# 3. Load by role: build one SELECT per AHU from the map,
        and stack them with UNION ALL.
selects, params = [], {}
for i, (equip, table) in enumerate(EQUIPMENT.items()):
    real = {row[1] for row in conn.execute(f'PRAGMA
        table_info("{table}")')}
    cols = {name: quote_ident(col_map[equip][role], real)
        for role, name in ROLES.items()}
    params[f"eq{i}"] = equip  # a value, so it is bound
    selects.append(
        f"SELECT :eq{i} AS equipment_id, timestamp_utc, "
        + ", ".join(f"{cols[name]} AS {name}" for name in
        ROLES.values())
        + f' FROM "{table}"'
    )
OA_SQL = "CREATE TEMP TABLE oa AS " + "\nUNION
        ALL\n".join(selects)
conn.execute(OA_SQL, params)

# 4. One summary for both AHUs: only rows where the supply
        fan runs
SUMMARY_SQL = """
SELECT o.equipment_id, d.damper_column,
       count(*)                                  AS
         run_rows,
       count(*) * 5 / 60.0                       AS
         run_hours,
       avg(o.oa_damper_pct)                      AS
         mean_damper_pct,
```

```
        sum(o.oa_damper_pct > 50) * 5 / 60.0        AS
          open_over_50_h,
        sum(o.oa_damper_pct <= 0) * 5 / 60.0        AS
          closed_h
FROM oa o
JOIN (SELECT :e1 AS equipment_id, :c1 AS damper_column
      UNION ALL SELECT :e2, :c2) d ON d.equipment_id =
          o.equipment_id
WHERE o.fan_status = 1
GROUP BY o.equipment_id
ORDER BY o.equipment_id
"""
dc = dict(zip(damper_columns.equipment_id,
          damper_columns.damper_column, strict=True))
summary = pd.read_sql_query(
    SUMMARY_SQL, conn, params={"e1": "AHU_1", "c1":
          dc["AHU_1"], "e2": "AHU_2", "c2": dc["AHU_2"]}
)

RESULTS = {
    "hardcoded_worked": hardcoded[["equipment_id",
          "worked"]],
    "damper_columns": damper_columns,
    "damper_summary": summary,
}

print("Hard-coded `mad_c` on each AHU:")
print(hardcoded.to_string(index=False))
print("\nDamper column per AHU, from column_map.json:")
print(damper_columns.to_string(index=False))
print("\nThe SQL built from the map:")
print(OA_SQL)
print("\nOutside-air damper while the supply fan runs (one
          query, both AHUs):")
print(summary.round(2).to_string(index=False))
```

Day 22 — Haystack tags as RDF: find a rule's inputs by tags with SPARQL

Goal: read one point's Haystack tags in `model/building_50.ttl`, then find FC1's three inputs for every AHU by asking for tags, in SPARQL (rdflib, pandas side) and in a flat SQL table (SQL side), and see what the graph holds that the table doesn't.

Read

Project Haystack describes equipment and points with **tags**. A **marker** tag is just a name (`point`, `sensor`, `pressure`, `his`); a **valued** tag has a value (`unit` = `inH2O`, `kind` = `Number`). Two valued tags are **refs**, pointers to other entities: `equipRef` ("this point belongs to that equipment") and `siteRef`. Tags are how people in the industry describe buildings, and this repo writes them as RDF, in its own `bts:` vocabulary, so a graph database can query them. One point, `bldg:AHU_1_DA_P`, in Turtle:

```
bldg:AHU_1_DA_P a bts:Entity ;
    rdfs:label "AHU1 DA-P" ;
    ref:hasExternalReference [ ... ] ;
# where its history lives (Day 24)
    bts:hasHaystackMarker "air", "discharge", "his",
"point", "pressure", "sensor" ;
    bts:hasHaystackTag [ bts:tagName "equipRef" ;
bts:tagValue bldg:AHU_1 ],
                       [ bts:tagName "unit" ;
bts:tagValue "inH2O" ],
                       [ bts:tagName "ofddRole" ;
bts:tagValue "duct-static-pressure" ], ... .
```

Markers are one predicate with many values. Valued tags are small nodes with a `tagName` and a `tagValue`. `ofddRole` is this repo's tag for the Open-FDD role: the same role names as Day 21's map, now stored *in the model*, on the point. `equipRef`'s value is a node (`bldg:AHU_1`), not a string, so the graph can follow it.

Finding FC1's inputs. FC1 needs the fan status, duct static pressure and its setpoint, for each AHU. In SPARQL that is a pattern of tags with a `FILTER` on the role:

```
SELECT ?equip ?role ?point ?timeseries_id WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseries_id .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
"duct-static-pressure-sp"))
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
.
}
```

Pattern order matters. rdflib evaluates triple patterns roughly top to bottom. The first pattern starts from the 118 points that have a timeseries reference, so every join after it is small. Start with "every node that has an `ofddRole` tag" and rdflib builds a large intermediate result first; against the API that hit its 10 s limit. A database planner reorders joins for you; a simple engine does not, so put the most selective pattern first.

The flat shadow. The model was built from `model/points/*.csv`, one row per point (point id, equipment, role, source column, UUID, …). The SQL script loads those rows into a `TEMP` table `point` and asks the same question as a plain `WHERE openfdd_role IN (...)`. The same three points per AHU come back. The SQL version is *simpler*. So why have a graph?

	flat table `point`	**RDF graph**
a point's role	a column, `openfdd_role`	a tag: `ofddRole`
which equipment	a column, derived here from the *file name*	a tag, `equipRef`, pointing at an entity
markers (`sensor`, `pressure`, …)	none: the CSV never had them	a first-class predicate

	flat table `point`	RDF graph
hierarchy (point → equipment → site)	one `owner_id` column, one level	refs you can follow to any depth
where the history lives	one column `timeseries_id`, one store	any number of references, each with its own store (Day 24)
a new kind of fact	`ALTER TABLE`	add a triple

The table is a **flat shadow**: it answers today's question fast, but it only contains what was flattened into it. The graph keeps the structure.

Step	`day22_pandas.py` (rdflib)	`day22_sql.py` (SQLite)
load	`Graph().parse("building_50.ttl")` (4,147 triples)	`INSERT` 118 CSV rows into `TEMP TABLE point`
FC1's inputs	SPARQL `FILTER(?role IN ...)`	`WHERE openfdd_role IN (...)`
per-AHU count	`GROUP BY ?equip`	`GROUP BY equipment_id`

Run

```
uv run python lessons/course.py run 22
```

That runs `day22_pandas.py`, then `day22_sql.py`. No server is involved: rdflib reads the Turtle file straight into memory. Both end with the same six rows (the pandas script also prints one point's markers and tags, which the table doesn't have):

```
equipment_id                    role        point_id
timeseries_id
       AHU_1    duct-static-pressure      AHU_1_DA_P
52422584-9894-50bd-9696-2fbb8b062518
       AHU_1 duct-static-pressure-sp   AHU_1_DA_P_SP
761c2b61-4982-5157-bc66-2010cf6e2a47
       AHU_1              fan-status AHU_1_SF_Status
9ebf80dd-7821-510b-b2c0-cd0151e093c9
       AHU_2    ...
```

Nothing in either script names a column. The two are compared on `(equipment, role, point id)` plus the UUID, and on the number of role-tagged points per AHU (13 each).

Try it

1. Find the AHUs' **discharge air temperature** and its setpoint (`discharge-air-temp`, `discharge-air-temp-sp`) in both scripts. How many lines change?
2. In the pandas script, query by *marker* instead of role: which points have the markers `pressure` **and** `sp`? Can the SQL table answer that?

Answers

1. Only the role strings in the filter (`FILTER(?role IN (...))` and `WHERE openfdd_role IN (...)`).
2. Two marker values on the same point: `?p bts:hasHaystackMarker "pressure", "sp" .` (the comma repeats the same subject and predicate). The flat table cannot: the CSVs have no markers. You could add a `markers` column, but then "has marker X" is a string search, where the graph has it as data.

Why it matters

Rules that ask for roles survive a change of building, as long as the building's model has the tags. That is why Haystack tagging is worth the effort in industry. Note what is still missing: the answer is a UUID, an opaque string you can't read. Tomorrow we look at where it comes from and at the table that stores history under it.

day22_pandas.py

```python
"""Day 22, pandas: find FC1's inputs by Haystack tags,
         with SPARQL on the RDF model (rdflib, no
server).

    uv run python lessons/unit3-modeling/day22_pandas.py

model/building_50.ttl holds every point as Haystack tags
         written as RDF (markers, ofddRole,
equipRef...). rdflib loads it into memory and runs SPARQL;
         the result rows become a DataFrame.
Nothing here names a column or an id: the query asks for
         roles, the model answers. Compare the
printout with day22_sql.py: the same (equipment, role,
         point) rows come back."""

import unicodedata
from pathlib import Path

import pandas as pd
from rdflib import Graph

ROOT = Path(__file__).resolve().parents[2]
MODEL = ROOT / "model" / "building_50.ttl"

PREFIXES = """
PREFIX bts:  <https://example.org/openfdd/bldgts#>
PREFIX bldg: <https://example.org/openfdd/BUILDING_50#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ref:  <https://brickschema.org/schema/Brick/ref#>
"""


def local(iri) -> str:
    """The part of an IRI after '#': bldg:AHU_1 ->
         'AHU_1'."""
    return str(iri).rsplit("#", 1)[-1]


def plain(text) -> str:
    """NFKD turns the unit's subscript 2 into a plain 2,
         so any console can print the tag."""
    return unicodedata.normalize("NFKD", local(text))


# 1. Load the model: a graph of (subject, predicate,
         object) statements, held in memory.
g = Graph()
```

```
g.parse(MODEL, format="turtle")

# 2. What does one point look like? Its markers, then its
        tags (name, value) as RDF.
MARKERS_SPARQL = PREFIXES + "SELECT ?marker WHERE {
        bldg:AHU_1_DA_P bts:hasHaystackMarker ?marker }"
TAGS_SPARQL = (
    PREFIXES
    + """
SELECT ?tag_name ?tag_value WHERE {
  bldg:AHU_1_DA_P bts:hasHaystackTag ?t .
  ?t bts:tagName ?tag_name ; bts:tagValue ?tag_value .
}"""
)
markers = sorted(str(r.marker) for r in
        g.query(MARKERS_SPARQL))
one_point = pd.DataFrame(
    [(str(r.tag_name), plain(r.tag_value)) for r in
        g.query(TAGS_SPARQL)],
    columns=["tag_name", "tag_value"],
).sort_values("tag_name", ignore_index=True)

# 3. FC1's inputs per AHU, by tags. Start from the points
        that have a timeseries reference (118 of
#    them): rdflib evaluates patterns roughly in order, so
        a selective first pattern keeps every
#    join after it small. Starting from "every ofddRole
        tag" is much slower.
FC1_ROLES = ("fan-status", "duct-static-pressure", "duct-
        static-pressure-sp")
FC1_SPARQL = (
    PREFIXES
    + """
SELECT ?equip ?role ?point ?timeseries_id WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseries_id .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
        "duct-static-pressure-sp"))
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
        .
}"""
)
fc1_inputs = pd.DataFrame(
    [
        (local(r.equip), str(r.role), local(r.point),
         str(r.timeseries_id))
        for r in g.query(FC1_SPARQL)
```

```
    ],
    columns=["equipment_id", "role", "point_id",
        "timeseries_id"],
).sort_values(["equipment_id", "role"], ignore_index=True)

# 4. How many points per AHU carry a fault-rule role at
    all?
ROLE_COUNT_SPARQL = (
    PREFIXES
    + """
SELECT ?equip (COUNT(?role) AS ?roles) WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseries_id .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
    .
}
GROUP BY ?equip"""
)
roles_per_ahu = pd.DataFrame(
    [(local(r.equip), int(r.roles)) for r in
        g.query(ROLE_COUNT_SPARQL)],
    columns=["equipment_id", "roles"],
).sort_values("equipment_id", ignore_index=True)

RESULTS = {"fc1_inputs": fc1_inputs, "roles_per_ahu":
    roles_per_ahu, "one_point_tags": one_point}

print(f"The graph has {len(g)} statements.\n")
print("Markers of bldg:AHU_1_DA_P:", ", ".join(markers))
print("\nIts tags:")
print(one_point.to_string(index=False))
print("\nFC1's inputs, found by role and equipRef:")
print(fc1_inputs.to_string(index=False))
print("\nPoints with a fault-rule role, per AHU:")
print(roles_per_ahu.to_string(index=False))
```

day22_sql.py

```
"""Day 22, SQL: find FC1's inputs by role, in a flat point
        table (a shadow of the RDF graph).

    uv run python lessons/unit3-modeling/day22_sql.py

model/points/*.csv is the point list the RDF model was
        built from, one row per point. It goes into
a TEMP table, and the same question is asked of it: which
        points play
FC1's roles? SQLite does the work; pandas only collects
        the rows for printing. Compare the
printout with day22_pandas.py: the same (equipment, role,
        point) rows come back."""

import csv
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
POINT_CSVS = sorted((ROOT / "model" /
        "points").glob("*.csv"))
conn = sqlite3.connect(":memory:")  # nothing to set up:
        the table is built from the CSVs below

# 1. Load the flat shadow: one row per point. equipment_id
        comes from the file name
#    (BUILDING_50__AHU_1.csv -> AHU_1); everything else is
        a CSV column.
conn.execute(
    """CREATE TEMP TABLE point (
        timeseries_id TEXT PRIMARY KEY, point_id TEXT NOT
        NULL, equipment_id TEXT NOT NULL,
        owner_id TEXT, brick_class TEXT, unit TEXT,
        openfdd_role TEXT, source_column TEXT)"""
)
rows = []
for path in POINT_CSVS:
    equipment_id = path.stem.split("__", 1)[1]
    with open(path, encoding="utf-8", newline="") as f:
        for r in csv.DictReader(f):
            rows.append(
                (
                    r["timeseries_id"],
                    r["point_id"],
                    equipment_id,
                    r["owner_id"] or None,
```

```
                    r["brick_class"] or None,
                    r["unit"] or None,
                    r["openfdd_role"] or None,
                    r["source_column"],
                )
            )
conn.executemany("INSERT INTO point VALUES
        (?,?,?,?,?,?,?,?)", rows)
conn.execute("CREATE INDEX temp.point_role ON point
        (openfdd_role)")

# 2. What does one point look like? One row: the table has
        columns where the graph has tags.
#    (There are no markers here: the CSV never had them.
        The graph adds them.)
ONE_POINT_SQL = "SELECT * FROM point WHERE point_id =
        :point_id"
one_point = pd.read_sql_query(ONE_POINT_SQL, conn, params=
        {"point_id": "AHU_1_DA_P"})

# 3. FC1's inputs per AHU, by role: a WHERE on the role
        column.
FC1_SQL = """
SELECT equipment_id, openfdd_role AS role, point_id,
        timeseries_id
FROM point
WHERE openfdd_role IN ('fan-status', 'duct-static-
        pressure', 'duct-static-pressure-sp')
ORDER BY equipment_id, openfdd_role
"""
fc1_inputs = pd.read_sql_query(FC1_SQL, conn)

# 4. How many points per AHU carry a fault-rule role at
        all?
ROLE_COUNT_SQL = """
SELECT equipment_id, count(openfdd_role) AS roles
FROM point
GROUP BY equipment_id
ORDER BY equipment_id
"""
roles_per_ahu = pd.read_sql_query(ROLE_COUNT_SQL, conn)

RESULTS = {"fc1_inputs": fc1_inputs, "roles_per_ahu":
        roles_per_ahu}

print(f"The point table has {len(rows)} rows (one per
        point).\n")
print("One point, as a row:")
print(one_point.T.to_string(header=False))
print("\nFC1's inputs, found by role and equipment:")
print(fc1_inputs.to_string(index=False))
```

```
print("\nPoints with a fault-rule role, per AHU:")
print(roles_per_ahu.to_string(index=False))
```

Day 23 — UUIDs and the long table: stable ids, `samples`, and pivoting long to wide

Goal: know why the history is stored under opaque, stable ids, see where Building 50's UUIDs come from, and pivot AHU_1's three FC1 inputs from the long `samples` table back into the CSV's columns.

Read

Why an opaque id. In the CSV a point's identity is its column name, `da_p_inwc`. That is fragile: the column is renamed when a controller is replaced, two buildings use the same name for different things (Day 21), and merging two exports collides names. A model-driven system wants an id with no meaning that never changes, so *everything else* (names, labels, units, tags) can change around it. Day 21's `mad_c` / `mad_c_pct` mismatch can't happen to an id nobody reads.

Where these ids come from. Each `timeseries_id` is a **UUIDv5**, defined in RFC 9562 (the 2024 update of RFC 4122): the SHA-1 hash of a namespace UUID plus a name, formatted as a UUID. The same name always gives the same id, on any machine, with no registry to ask, so rebuilds and merges agree. This repo's rule:

```
uuid.uuid5(uuid.NAMESPACE_URL, f"urn:openfdd:{building}:
        {equipment}:{source_column}")
# urn:openfdd:BUILDING_50:AHU_1:da_p_inwc  ->  52422584-
        9894-50bd-9696-2fbb8b062518
```

Both scripts recompute the id and compare it with the one in the point list. There is one trap: the id is derived from the *name once*, then **frozen** in `model/points/*.csv`. If you re-derived it at query time and someone renamed the column, the id would change and every stored series would be orphaned. The hash is a way to *mint* ids reproducibly, not a way to *find*

them later. (A v4 UUID, random, would also work. You'd just have to keep the table that says which one is which, which is exactly what the frozen CSV is.)

The long table. The API's database, `var/timeseries.sqlite`, has no column names at all:

```
CREATE TABLE samples (timeseries_id TEXT, ts INTEGER,
        value REAL,
                    PRIMARY KEY (timeseries_id, ts))
        WITHOUT ROWID
```

That is **long** (tall) form: one row per reading, 4.2 million rows for 118 points. The CSV is **wide**: one row per timestamp, one column per point. Long form has real advantages for storage. A new point is new rows, not a new column (`ALTER TABLE`); points that report at different times or with gaps don't need NULL padding; and the primary key `(timeseries_id, ts)` with `WITHOUT ROWID` stores each series contiguously and in time order, so "all of one series between two times" is one B-tree range scan. Wide form is what rules want: several signals side by side at the same timestamp.

Going from long to wide is a **pivot**: one output row per timestamp, one output column per series.

Step	`day23_pandas.py`	`day23_sql.py`
role → UUID	`read_csv(model/points/...)`	`TEMP TABLE point` from tl same CSV
recompute the UUID	`uuid.uuid5(...)`	a Python function register in SQLite (`create_function`): SQLit has no SHA-1
read the long rows	`pd.read_sql_query("... WHERE timeseries_id IN (?,?,?)")`	`FROM samples s JOIN point p ON p.timeseries_id = s.timeseries_id`
pivot	`long.pivot(index="ts", columns="column", values="value")`	`MAX(CASE WHEN p.col = 'fan_status' THEN s.value END) ... GROUP BY s.ts`

Step	day23_pandas.py	day23_sql.py
compare to the CSV	merge on epoch seconds, == per column	JOIN les.ahu_1 ON unixepoch(timestamp_ut = w.ts, from an ATTACHed second database

`MAX(CASE ...)` is conditional aggregation again: within one timestamp each series has exactly one row, so `MAX` just picks it and the other rows' NULLs are ignored. pandas' `pivot` is stricter and raises if a `(ts, series)` pair repeats, which is a free check that your data has one value per timestamp. Either way it is O(n): one pass over the long rows, then a group per timestamp.

The wide result has to equal the CSV's own columns, because the database was filled from that CSV. The scripts check it value by value, and show the number of equal rows and the largest absolute difference.

Run

This day needs `var/timeseries.sqlite`: `uv run bldgts model build && uv run bldgts bootstrap` once. (The test suite skips the day if the file is missing.) The scripts open it read-only.

```
uv run python lessons/course.py run 23
```

That runs `day23_pandas.py`, then `day23_sql.py`. Both print:

```
The UUID of each FC1 input, and the same UUID minted again
from its name:
 ... 52422584-9894-50bd-9696-2fbb8b062518 52422584-9894-
50bd-9696-2fbb8b062518   True
Long table: 106,608 rows for 3 series.
Wide table: 35,536 rows, one per timestamp.
                  role      source_column  rows_compared
rows_equal  max_abs_diff
   duct-static-pressure          da_p_inwc          35536
35536           0.0
duct-static-pressure-sp da_p_setpoint_inwc          35536
35536           0.0
             fan-status  supply_fan_status          35536
35536           0.0
```

(The SQL script prints `1` where pandas prints `True`; the tests treat them as equal.) 3 series × 35,536 rows = 106,608 long rows become 35,536 wide rows.

Try it

1. Change one letter of the name (`da_p_inwcX`) and mint the id again. How different is the result? What does that tell you about hand-editing ids?
2. Add the outside-air temperature to the pivot (role `outside-air-temp`) in both scripts. What does the long table cost per extra point, compared with adding a CSV column?

Answers

1. Completely different: a hash has no “nearby” ids. So an id can’t be fixed by hand, and a rename means a new series unless you keep the old id. That is why the id is frozen in the point list.
2. One more `FROM`/`WHERE` match on a UUID, and no schema change at all. The long table just has more rows. That is the storage argument for long form: points come and go without altering the table.

Why it matters

Now the model's answer ("the fan status is `9ebf80dd-...`") leads to a series, and you can rebuild the wide table a rule wants from the long one. The next question: the model says *an id*, but what says *where to look it up*? Tomorrow's answer is the external timeseries reference.

day23_pandas.py

```python
"""Day 23, pandas: stable UUIDs, the long `samples` table,
         and pivoting AHU_1's FC1 inputs to wide.

    uv run python lessons/unit3-modeling/day23_pandas.py

Needs var/timeseries.sqlite (the API's database: `uv run
         bldgts model build && uv run bldgts
bootstrap`). It stores history *long*: one row per reading
         in samples(timeseries_id, ts, value),
with no column names. The point list says which UUID is
         which role. Pivot long -> wide and the
result must equal the CSV columns. Compare the printout
         with day23_sql.py: the numbers should
match."""

import sqlite3
import uuid
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
POINTS = ROOT / "model" / "points" /
         "BUILDING_50__AHU_1.csv"
SAMPLES_DB = ROOT / "var" / "timeseries.sqlite"
if not SAMPLES_DB.exists():
    raise SystemExit(
        f"{SAMPLES_DB} not found. Run:  uv run bldgts
         model build && uv run bldgts bootstrap"
    )

BUILDING, EQUIP = "BUILDING_50", "AHU_1"
ROLE_NAMES = {  # FC1's input contract: role -> the column
         name the rule uses
    "fan-status": "fan_status",
    "duct-static-pressure": "duct_static_inwc",
    "duct-static-pressure-sp": "duct_static_sp_inwc",
}

# 1. The point list: which UUID is which role, and which
         CSV column it came from.
pts = pd.read_csv(POINTS, usecols=["openfdd_role",
         "source_column", "timeseries_id"])
pts = pts[pts["openfdd_role"].isin(ROLE_NAMES)].copy()
pts["column"] = pts["openfdd_role"].map(ROLE_NAMES)
pts = pts.rename(columns={"openfdd_role":
         "role"}).sort_values("role", ignore_index=True)
```

```
# 2. Where did those UUIDs come from? A UUIDv5 (RFC 9562)
        is a SHA-1 hash of a namespace and a name.
#    The same name always gives the same id, on any
        machine, with no registry. Build's rule:
def mint(building: str, equipment: str, source_column:
        str) -> str:
    return str(
        uuid.uuid5(uuid.NAMESPACE_URL, f"urn:openfdd:
        {building}:{equipment}:{source_column}")
    )


pts["recomputed_uuid"] = [mint(BUILDING, EQUIP, c) for c
        in pts["source_column"]]
uuid_check = pts[["role", "source_column",
        "timeseries_id", "recomputed_uuid"]].assign(
    matches=pts["timeseries_id"] == pts["recomputed_uuid"]
)

# 3. The long table: one row per (timeseries_id, ts). Only
        the three UUIDs we asked the model for.
marks = ",".join("?" * len(pts))
conn = sqlite3.connect(f"file:{SAMPLES_DB.as_posix()}?
        mode=ro", uri=True)
long = pd.read_sql_query(
    f"SELECT timeseries_id, ts, value FROM samples WHERE
        timeseries_id IN ({marks})",
    conn,
    params=list(pts["timeseries_id"]),
)
long = long.merge(pts[["timeseries_id", "role",
        "column"]], on="timeseries_id")
long_counts = (
    long.groupby("role")
    .agg(rows=("value", "size"), first_ts=("ts", "min"),
        last_ts=("ts", "max"))
    .reset_index()
)

# 4. Long -> wide: one row per timestamp, one column per
        role. pivot() raises if a (ts, role) pair
#    repeats, which is a useful check that the series
        really have one value per timestamp.
wide = long.pivot(index="ts", columns="column",
        values="value").reset_index()
wide.columns.name = None
wide = wide[["ts",
        *ROLE_NAMES.values()]].sort_values("ts",
        ignore_index=True)
```

```
# 5. Compare with the CSV the history came from: same
        timestamps, same values?
csv_cols = list(pts["source_column"])
csv = pd.read_csv(DATA / EQUIP / "history_wide.csv",
        usecols=["timestamp_utc", *csv_cols])
csv["ts"] = pd.to_datetime(csv["timestamp_utc"],
        utc=True).dt.as_unit("s").astype("int64")
both = csv.merge(wide, on="ts", how="inner")
csv_match = pd.DataFrame(
    [
        {
            "role": r.role,
            "source_column": r.source_column,
            "rows_compared": len(both),
            "rows_equal": int((both[r.source_column] ==
         both[r.column]).sum()),
            "max_abs_diff": float((both[r.source_column] -
         both[r.column]).abs().max()),
        }
        for r in pts.itertuples()
    ]
)

RESULTS = {
    "uuid_check": uuid_check,
    "long_counts": long_counts,
    "wide": wide,
    "csv_match": csv_match,
}

print("The UUID of each FC1 input, and the same UUID
        minted again from its name:")
print(uuid_check.to_string(index=False))
print(f"\nLong table: {len(long):,} rows for {len(pts)}
        series.")
print(long_counts.to_string(index=False))
print(f"\nWide table: {len(wide):,} rows, one per
        timestamp. First rows:")
print(wide.head(3).to_string(index=False))
print("\nCompared with the CSV's own columns:")
print(csv_match.to_string(index=False))
```

day23_sql.py

```python
"""Day 23, SQL: stable UUIDs, the long `samples` table, and pivoting AHU_1's FC1 inputs to wide.

    uv run python lessons/course.py setup          # once: var/lessons.sqlite (the CSVs)
    uv run python lessons/unit3-modeling/day23_sql.py

Needs var/timeseries.sqlite (the API's database: `uv run bldgts model build && uv run bldgts bootstrap`). The same computation as day23_pandas.py, done by SQLite: the point list becomes a TEMP table, long -> wide is MAX(CASE ...) and the CSV comparison is a join across two attached database files. Compare the printout with day23_pandas.py: the numbers should match."""

import csv
import os
import sqlite3
import uuid
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
POINTS = ROOT / "model" / "points" / "BUILDING_50__AHU_1.csv"
SAMPLES_DB = ROOT / "var" / "timeseries.sqlite"
LESSONS_DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" / "lessons.sqlite"))
if not SAMPLES_DB.exists():
    raise SystemExit(
        f"{SAMPLES_DB} not found. Run:  uv run bldgts model build && uv run bldgts bootstrap"
    )
if not LESSONS_DB.exists():
    raise SystemExit(f"{LESSONS_DB} not found. Run:  uv run python lessons/course.py setup")

ROLE_NAMES = {  # FC1's input contract: role -> the column name the rule uses
    "fan-status": "fan_status",
    "duct-static-pressure": "duct_static_inwc",
    "duct-static-pressure-sp": "duct_static_sp_inwc",
}
```

```
def mint(building: str, equipment: str, source_column:
        str) -> str:
    """UUIDv5 (RFC 9562): SHA-1 of a namespace and a name.
        SQLite has none, so lend it Python's."""
    return str(
        uuid.uuid5(uuid.NAMESPACE_URL, f"urn:openfdd:
        {building}:{equipment}:{source_column}")
    )


def quote_ident(name: str, allowed: set[str]) -> str:
    """Identifiers can't be bound (Day 21): accept a name
        that really exists, then quote it."""
    if name not in allowed:
        raise ValueError(f"unknown column {name!r}")
    return '"' + name.replace('"', '""') + '"'


# Open the API's database read-only; attach the lessons
        database (the CSV rows) next to it.
conn = sqlite3.connect(f"file:{SAMPLES_DB.as_posix()}?
        mode=ro", uri=True)
conn.execute("ATTACH DATABASE ? AS les", (f"file:
        {LESSONS_DB.as_posix()}?mode=ro",))
conn.create_function("uuid5_urn", 3, mint,
        deterministic=True)

# 1. The point list: which UUID is which role, and which
        CSV column it came from.
building_id, equipment_id = POINTS.stem.split("__", 1)
conn.execute(
    """CREATE TEMP TABLE point (building_id TEXT,
        equipment_id TEXT, role TEXT,
                                source_column TEXT,
        timeseries_id TEXT PRIMARY KEY, col TEXT)"""
)
with open(POINTS, encoding="utf-8", newline="") as f:
    conn.executemany(
        "INSERT INTO point VALUES (?,?,?,?,?,?)",
        [
            (
                building_id,
                equipment_id,
                r["openfdd_role"],
                r["source_column"],
                r["timeseries_id"],
                ROLE_NAMES[r["openfdd_role"]],
            )
            for r in csv.DictReader(f)
            if r["openfdd_role"] in ROLE_NAMES
```

```
        ],
    )

# 2. Where did those UUIDs come from? Mint them again from
     their names and compare.
UUID_SQL = """
SELECT role, source_column, timeseries_id,
       uuid5_urn(building_id, equipment_id, source_column)
         AS recomputed_uuid,
       timeseries_id = uuid5_urn(building_id,
         equipment_id, source_column) AS matches
FROM point
ORDER BY role
"""
uuid_check = pd.read_sql_query(UUID_SQL, conn)

# 3. The long table: one row per (timeseries_id, ts). The
     primary key (timeseries_id, ts) makes
#    "all of one series" a range scan, and WITHOUT ROWID
     stores the rows in that order.
LONG_SQL = """
SELECT p.role, count(*) AS rows, min(s.ts) AS first_ts,
       max(s.ts) AS last_ts
FROM samples s
JOIN point p ON p.timeseries_id = s.timeseries_id
GROUP BY p.role
ORDER BY p.role
"""
long_counts = pd.read_sql_query(LONG_SQL, conn)

# 4. Long -> wide: one row per timestamp, one column per
     role (conditional aggregation, Day 9).
WIDE_SQL = "CREATE TEMP TABLE wide (ts INTEGER PRIMARY
     KEY, {cols_def}) WITHOUT ROWID"
PIVOT_SQL = """
INSERT INTO wide
SELECT s.ts, {cols}
FROM samples s
JOIN point p ON p.timeseries_id = s.timeseries_id
GROUP BY s.ts
"""
names = list(ROLE_NAMES.values())
conn.execute(WIDE_SQL.format(cols_def=", ".join(f"{n}
     REAL" for n in names)))
conn.execute(
    PIVOT_SQL.format(
        cols=", ".join(f"MAX(CASE WHEN p.col = '{n}' THEN
     s.value END)" for n in names)
    )
)
```

```
wide = pd.read_sql_query("SELECT * FROM wide ORDER BY ts",
          conn)

# 5. Compare with the CSV the history came from (table
          les.ahu_1): same timestamps, same values?
#    The CSV column names come from the point list, so
          they are whitelisted and quoted.
real = {row[1] for row in conn.execute("PRAGMA
          les.table_info(ahu_1)")}
role_rows = conn.execute("SELECT role, source_column, col
          FROM point ORDER BY role").fetchall()
MATCH_SQL = """
SELECT :role AS role, :source_column AS source_column,
       count(*) AS rows_compared,
       sum(c.{src} = w.{col}) AS rows_equal,
       max(abs(c.{src} - w.{col})) AS max_abs_diff
FROM les.ahu_1 c
JOIN wide w ON w.ts = unixepoch(c.timestamp_utc)
"""
parts = [
    pd.read_sql_query(
        MATCH_SQL.format(src=quote_ident(src, real),
         col=quote_ident(col, set(names))),
        conn,
        params={"role": role, "source_column": src},
    )
    for role, src, col in role_rows
]
csv_match = pd.concat(parts, ignore_index=True)

RESULTS = {
    "uuid_check": uuid_check,
    "long_counts": long_counts,
    "wide": wide,
    "csv_match": csv_match,
}

print("The UUID of each FC1 input, and the same UUID
          minted again from its name:")
print(uuid_check.to_string(index=False))
print(f"\nLong table: {int(long_counts['rows'].sum()):,}
          rows for {len(long_counts)} series.")
print(long_counts.to_string(index=False))
print(f"\nWide table: {len(wide):,} rows, one per
          timestamp. First rows:")
print(wide.head(3).to_string(index=False))
print("\nCompared with the CSV's own columns:")
print(csv_match.to_string(index=False))
```

Day 24 — External timeseries references: one model, history in SQLite or in a CSV

Goal: see how the model says *where* a point's history lives, add a second kind of store (a CSV file) to an in-memory copy of the graph, write a resolver that reads a series from whichever store a reference names, and check that both stores give the same series.

Read

What plain Haystack leaves out. Haystack marks a point as having history with the marker `his` and reads it by point id (the `hisRead` operation: "give me the history of point X"). But it has no standard tag for *where* that history is stored. Which database? Which table, key, file? That is left to the server you ask.

What the ref schema adds. Brick's **ref schema** (the page "Timeseries storage" in the Brick docs) fills exactly that gap, and it doesn't care what the store is. A point has a `ref:hasExternalReference` to a `ref:TimeseriesReference`. The reference has exactly one `ref:hasTimeseriesId` (a string: the key of the series in the store) and a `ref:storedAt` pointing at the store, which Brick types as a `ref:Database`. Brick doesn't say what a database node must contain. This repo combines the two vocabularies, Haystack tags for *what the point is* and reference nodes for *where its data is*:

```
bldg:AHU_1_DA_P  bts:hasHaystackMarker "his", "point", ...
;
    ref:hasExternalReference [ a ref:TimeseriesReference ;
        ref:hasTimeseriesId "52422584-9894-50bd-9696-
2fbb8b062518" ;
        ref:storedAt bldg:timeseries_db ] .

bldg:timeseries_db a ref:Database ;
    bts:backend "sqlite" ;  bts:connectionEnv
"BLDGTS_DB_PATH" ;
    bts:schemaDescription "samples(timeseries_id TEXT, ts
INTEGER epoch-s UTC, value REAL)" .
```

So the model has, for every point, "ask `timeseries_db` (SQLite; path in `$BLDGTS_DB_PATH`) for id `52422584-...`". Because the reference is a node, a point can have **several**: the same sensor's history in the live database and in an archive, or in a file someone sent you.

A second store. Today we add one, in memory, without touching `model/building_50.ttl`. Each AHU's CSV export becomes a store, and each point gets a second reference to it. A CSV has no UUIDs: a series is identified by its column, so that CSV reference's `hasTimeseriesId` is the **column name** (`da_p_inwc`). Where does the script learn which column goes with which role? From Day 21's `column_map.json`. The model's roles plus the map give the column.

```
bldg:csv_AHU_1 a bts:CsvFile ;  bts:path
"data/BUILDING_50/AHU_1/history_wide.csv" ;
                                bts:timeColumn
"timestamp_utc" .
bldg:AHU_1_DA_P ref:hasExternalReference [ a
ref:TimeseriesReference ;
        ref:hasTimeseriesId "da_p_inwc" ; ref:storedAt
bldg:csv_AHU_1 ] .
```

The store gets its own class, `bts:CsvFile`, and not `ref:Database`. Brick's `ref:Database` represents a storage *system*; calling a file a "database" would stretch that class's meaning, and anything that dispatches on the class ("a database: connect to it") would be wrong about a file. A class of our own keeps the dispatch honest.

The resolver is the one function that knows about stores. Given a reference it looks at the store's class and reads the series from the right place:

Store class	`day24_pandas.py`	`day24_sql.py`
`ref:Database` (sqlite)	`read_sql_query("SELECT ts, value FROM samples WHERE timeseries_id = ?")`	the same `SELECT`, against an `ATTACH`ed database
`bts:CsvFile`	`read_csv(path, usecols=[time, column])`, time text → epoch seconds	load the needed columns into a `TEMP` table, `unixepoch(timestamp_utc` as `ts`
anything else	`raise ValueError`	`raise ValueError`

Notice what stays the same: the rule asks the model for "the fan status of AHU_1" and gets back references; the resolver picks the store. Nothing in the rule knows whether the data came from a database or a file. Swapping storage means changing the model (and, for a brand-new store *type*, adding a branch to the resolver), not the rules.

One gotcha. Once a point has two references, a query that joins through `ref:hasExternalReference` returns that point *twice* (once per reference). The scripts use `SELECT DISTINCT ?point` to find the points and then ask for their references separately. If a result suddenly has twice the rows after you add a reference, this is why.

The scripts compare the two stores for each of the 6 FC1 input series (3 per AHU): same number of rows, same values?

Run

Needs `var/timeseries.sqlite` (`uv run bldgts model build && uv run bldgts bootstrap`).

```
uv run python lessons/course.py run 24
```

That runs `day24_pandas.py`, then `day24_sql.py`. Both build the same graph with rdflib (SQL can't read RDF), then differ in how they read the series. Both print:

```
       point_id         store store_class
timeseries_id
     AHU_1_DA_P     csv_AHU_1     CsvFile
da_p_inwc
     AHU_1_DA_P timeseries_db    Database 52422584-9894-
50bd-9696-2fbb8b062518
     ...
       point_id  rows_sqlite  rows_csv  rows_equal
max_abs_diff
     AHU_1_DA_P        35536     35536       35536
0.0
     ...
```

All six series have 35,536 rows in each store and every value is identical.

Try it

1. Add a third reference to one point: another `ref:Database`-typed node, with `bts:backend "postgres"`. What does the resolver do when it meets it? Is that the right behaviour?
2. Make the resolver pick the store: add a parameter `prefer` ("Database" or "CsvFile") that chooses among a point's references. What does a rule need to know to use it?

Answers

1. `read_series` raises `ValueError: no reader for database backend 'postgres'`. That is the right behaviour: failing loudly beats returning an empty series. To support it you add one branch (and a driver).
2. Nothing: `prefer` is part of the resolver call, not of the rule. Today's `refs` table has `store_class`; filter it before `read_series`. Day 25's challenge does exactly this.

Why it matters

This is why model-driven FDD survives a change of storage. When history moves from CSV to SQLite, or SQLite to a real time-series database, you change the *references*. The rules, written against roles and tags, don't move. Tomorrow's rule runs through the app's API, which does the resolving for us.

day24_pandas.py

```python
"""Day 24, pandas: external timeseries references, with
        the history in SQLite *or* in a CSV file.

    uv run python lessons/unit3-modeling/day24_pandas.py

Needs var/timeseries.sqlite (`uv run bldgts model build &&
        uv run bldgts bootstrap`). Each point
in the model says where its history lives with a
        ref:TimeseriesReference: an id
(ref:hasTimeseriesId) and a store (ref:storedAt). The
        model already has one reference per point,
to the SQLite store. Here we add a second reference per
        point, to the AHU's CSV export, in an
in-memory copy of the graph. A small resolver reads a
        point's series from whichever store a
reference names, and we check both stores hold the same
        series. Compare the printout with
day24_sql.py: the numbers should match."""

import json
import os
import sqlite3
from pathlib import Path

import pandas as pd
from rdflib import RDF, BNode, Graph, Literal, Namespace,
        URIRef

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
MODEL = ROOT / "model" / "building_50.ttl"
AHUS = ("AHU_1", "AHU_2")

BTS = Namespace("https://example.org/openfdd/bldgts#")
BLDG = Namespace("https://example.org/openfdd/BUILDING_50#")
REF = Namespace("https://brickschema.org/schema/Brick/ref#")
PREFIXES = f"PREFIX bts: <{BTS}>\nPREFIX bldg:
        <{BLDG}>\nPREFIX ref: <{REF}>\n"


def local(iri) -> str:
    return str(iri).rsplit("#", 1)[-1]


# 1. Load the model. It was built with one reference per
        point, all to bldg:timeseries_db (SQLite).
g = Graph()
g.parse(MODEL, format="turtle")
```

```
# 2. Add a second kind of store: a CSV file. Plain
      Haystack has no tag for "where is the history";
#    the ref schema does, and it doesn't care what the
      store is. Brick's ref:Database means a
#    database, so we don't call a file one: we give CSV
      files their own class, bts:CsvFile, with
#    a path.
#    The "id" a CSV reference holds is the column name (a
      CSV has no UUIDs).
ROLE_POINTS_SPARQL = (
    PREFIXES
    + """
SELECT ?equip ?role ?point WHERE {
  ?point ref:hasExternalReference ?r .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
      .
}"""
)
role_points = [(r.equip, str(r.role), r.point) for r in
      g.query(ROLE_POINTS_SPARQL)]
col_map = {e: json.loads((DATA / e /
      "column_map.json").read_text())["points"] for e
      in AHUS}
for equip in AHUS:
    csv_store = BLDG[f"csv_{equip}"]
    g.add((csv_store, RDF.type, BTS.CsvFile))
    g.add((csv_store, BTS.path,
      Literal(f"data/BUILDING_50/{equip}/history_wide.csv")))
    g.add((csv_store, BTS.timeColumn,
      Literal("timestamp_utc")))
for equip_iri, role, point in role_points:
    column = col_map[local(equip_iri)].get(role)  # role -
      > column, from Day 21's first model
    if column is None:
        continue
    node = BNode()
    g.add((point, REF.hasExternalReference, node))
    g.add((node, RDF.type, REF.TimeseriesReference))
    g.add((node, REF.hasTimeseriesId, Literal(column)))
    g.add((node, REF.storedAt,
      BLDG[f"csv_{local(equip_iri)}"]))

# 3. Pick FC1's inputs by tag (Day 22) and list every
      reference each one has.
FC1_POINTS_SPARQL = (
    PREFIXES
    + """
```

```
SELECT DISTINCT ?point WHERE {
  ?point ref:hasExternalReference ?r .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
         "duct-static-pressure-sp"))
}"""
)
REFS_SPARQL = (
    PREFIXES
    + """
SELECT ?point ?ts_id ?store ?store_class WHERE {
  ?point ref:hasExternalReference ?r .
  ?r a ref:TimeseriesReference ; ref:hasTimeseriesId ?
         ts_id ; ref:storedAt ?store .
  ?store a ?store_class .
}"""
)
fc1_points = sorted(r.point for r in
         g.query(FC1_POINTS_SPARQL))
refs = pd.DataFrame(
    [
        (local(p), local(r.store), local(r.store_class),
         str(r.ts_id), str(r.store), str(p))
        for p in fc1_points
        for r in g.query(REFS_SPARQL, initBindings=
         {"point": p})
    ],
    columns=["point_id", "store", "store_class",
         "timeseries_id", "store_iri", "point_iri"],
).sort_values(["point_id", "store"], ignore_index=True)


# 4. The resolver: read one reference's series (ts in
         epoch seconds, value) from the store it names.
def read_series(store: str, store_class: str, ts_id: str)
         -> pd.DataFrame:
    node = URIRef(store)
    if store_class == "Database":  # ref:Database; the
         model says backend sqlite, path in an env var
        backend = str(g.value(node, BTS.backend))
        if backend != "sqlite":
            raise ValueError(f"no reader for database
         backend {backend!r}")
        env = str(g.value(node, BTS.connectionEnv))
        path = Path(os.environ.get(env, ROOT / "var" /
         "timeseries.sqlite"))
        conn = sqlite3.connect(f"file:{path.as_posix()}?
         mode=ro", uri=True)
        sql = "SELECT ts, value FROM samples WHERE
         timeseries_id = ? ORDER BY ts"
```

```
        df = pd.read_sql_query(sql, conn, params=[ts_id])
        conn.close()
        return df
    if store_class == "CsvFile":  # bts:CsvFile; the id is
         the column name
        time_col = str(g.value(node, BTS.timeColumn))
        df = pd.read_csv(ROOT / str(g.value(node,
         BTS.path)), usecols=[time_col, ts_id])
        ts = pd.to_datetime(df[time_col],
         utc=True).dt.as_unit("s").astype("int64")
        return pd.DataFrame({"ts": ts, "value":
         df[ts_id]})
    raise ValueError(f"no reader for store class
         {store_class!r}")


# 5. Resolve every reference of every FC1 point, and
       compare the two stores point by point.
series = {
    (r.point_id, r.store_class): read_series(r.store_iri,
         r.store_class, r.timeseries_id)
    for r in refs.itertuples()
}
rows = []
for point_id in sorted({p for p, _ in series}):
    db, csv = series[(point_id, "Database")],
         series[(point_id, "CsvFile")]
    both = db.merge(csv, on="ts", suffixes=("_db",
         "_csv"))
    rows.append(
        {
            "point_id": point_id,
            "rows_sqlite": len(db),
            "rows_csv": len(csv),
            "rows_equal": int((both["value_db"] ==
         both["value_csv"]).sum()),
            "max_abs_diff": float((both["value_db"] -
         both["value_csv"]).abs().max()),
        }
    )
series_compare = pd.DataFrame(rows)

RESULTS = {
    "refs": refs[["point_id", "store", "store_class",
         "timeseries_id"]],
    "series_compare": series_compare,
}

print("References of FC1's input points (a second store
         was added to each point in memory):")
print(RESULTS["refs"].to_string(index=False))
```

```
print("\nThe same series, read from SQLite and from the
        CSV file:")
print(series_compare.to_string(index=False))
```

day24_sql.py

```python
"""Day 24, SQL: external timeseries references, with the
        history in SQLite *or* in a CSV file.

    uv run python lessons/unit3-modeling/day24_sql.py

Needs var/timeseries.sqlite (`uv run bldgts model build &&
        uv run bldgts bootstrap`). The model
part is the same as day24_pandas.py (rdflib + SPARQL on an
        in-memory graph; a SQL database can't
read RDF). The resolver differs: for each reference it
        returns the SQL that reads that series from
the store the reference names, and SQLite compares the two
        stores. A CSV store is first loaded
into a TEMP table. Compare the printout with
        day24_pandas.py: the numbers should match."""

import csv
import json
import os
import sqlite3
from pathlib import Path

import pandas as pd
from rdflib import RDF, BNode, Graph, Literal, Namespace,
        URIRef

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
MODEL = ROOT / "model" / "building_50.ttl"
AHUS = ("AHU_1", "AHU_2")

BTS = Namespace("https://example.org/openfdd/bldgts#")
BLDG = Namespace("https://example.org/openfdd/BUILDING_50#")
REF = Namespace("https://brickschema.org/schema/Brick/ref#")
PREFIXES = f"PREFIX bts: <{BTS}>\nPREFIX bldg:
        <{BLDG}>\nPREFIX ref: <{REF}>\n"


def local(iri) -> str:
    return str(iri).rsplit("#", 1)[-1]


# 1. Load the model. It was built with one reference per
        point, all to bldg:timeseries_db (SQLite).
g = Graph()
g.parse(MODEL, format="turtle")
```

```
# 2. Add a second kind of store: a CSV file. Plain
      Haystack has no tag for "where is the history";
#    the ref schema does, and it doesn't care what the
      store is. Brick's ref:Database means a
#    database, so we don't call a file one: we give CSV
      files their own class, bts:CsvFile, with
#    a path.
#    The "id" a CSV reference holds is the column name (a
      CSV has no UUIDs).
ROLE_POINTS_SPARQL = (
    PREFIXES
    + """
SELECT ?equip ?role ?point WHERE {
  ?point ref:hasExternalReference ?r .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
      .
}"""
)
role_points = [(r.equip, str(r.role), r.point) for r in
      g.query(ROLE_POINTS_SPARQL)]
col_map = {e: json.loads((DATA / e /
      "column_map.json").read_text())["points"] for e
      in AHUS}
for equip in AHUS:
    csv_store = BLDG[f"csv_{equip}"]
    g.add((csv_store, RDF.type, BTS.CsvFile))
    g.add((csv_store, BTS.path,
      Literal(f"data/BUILDING_50/{equip}/history_wide.csv")))
    g.add((csv_store, BTS.timeColumn,
      Literal("timestamp_utc")))
for equip_iri, role, point in role_points:
    column = col_map[local(equip_iri)].get(role)  # role -
      > column, from Day 21's first model
    if column is None:
        continue
    node = BNode()
    g.add((point, REF.hasExternalReference, node))
    g.add((node, RDF.type, REF.TimeseriesReference))
    g.add((node, REF.hasTimeseriesId, Literal(column)))
    g.add((node, REF.storedAt,
      BLDG[f"csv_{local(equip_iri)}"]))

# 3. Pick FC1's inputs by tag (Day 22) and list every
      reference each one has.
FC1_POINTS_SPARQL = (
    PREFIXES
    + """
SELECT DISTINCT ?point WHERE {
```

```
  ?point ref:hasExternalReference ?r .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
         "duct-static-pressure-sp"))
}"""
)
REFS_SPARQL = (
    PREFIXES
    + """
SELECT ?point ?ts_id ?store ?store_class WHERE {
  ?point ref:hasExternalReference ?r .
  ?r a ref:TimeseriesReference ; ref:hasTimeseriesId ?
         ts_id ; ref:storedAt ?store .
  ?store a ?store_class .
}"""
)
fc1_points = sorted(r.point for r in
         g.query(FC1_POINTS_SPARQL))
refs = pd.DataFrame(
    [
        (local(p), local(r.store), local(r.store_class),
         str(r.ts_id), str(r.store), str(p))
        for p in fc1_points
        for r in g.query(REFS_SPARQL, initBindings=
         {"point": p})
    ],
    columns=["point_id", "store", "store_class",
         "timeseries_id", "store_iri", "point_iri"],
).sort_values(["point_id", "store"], ignore_index=True)


# 4. The resolver: for one reference, the SQL (and
         parameters) that read its series from the
#    store it names.
conn = sqlite3.connect(
    ":memory:", uri=True
)  # scratch connection; the samples database is attached
         read-only below
sqlite_stores = {}  # store IRI -> schema name of the
         attached SQLite database
csv_tables = {}  # store IRI -> name of the TEMP table
         holding that CSV file


def quote_ident(name: str, allowed: set[str]) -> str:
    """Identifiers can't be bound as parameters: accept a
         name that really exists, then quote it."""
    if name not in allowed:
        raise ValueError(f"unknown column {name!r}")
```

```
    return '"' + name.replace('"', '""') + '"'


def attach_sqlite(store: str) -> str:
    node = URIRef(store)
    backend = str(g.value(node, BTS.backend))
    if backend != "sqlite":
        raise ValueError(f"no reader for database backend
         {backend!r}")
    env = str(g.value(node, BTS.connectionEnv))
    path = Path(os.environ.get(env, ROOT / "var" /
         "timeseries.sqlite"))
    conn.execute("ATTACH DATABASE ? AS tsdb", (f"file:
         {path.as_posix()}?mode=ro",))
    return "tsdb"


def load_csv(store: str, columns: set[str]) -> str:
    """Load the columns a CSV store is asked for into a
         TEMP table (REAL values, epoch ts)."""
    node = URIRef(store)
    time_col = str(g.value(node, BTS.timeColumn))
    name = f"csv_{local(node)}"
    with open(ROOT / str(g.value(node, BTS.path)),
         encoding="utf-8", newline="") as f:
        reader = csv.reader(f)
        header = next(reader)
        wanted = [header.index(c) for c in
         sorted(columns)]  # raises if a column is missing
        t = header.index(time_col)
        cols = ", ".join(f"{quote_ident(c, set(header))}
         REAL" for c in sorted(columns))
        conn.execute(f'CREATE TEMP TABLE "{name}" (ts
         INTEGER PRIMARY KEY, {cols}) WITHOUT ROWID')
        marks = ", ".join("?" * (len(wanted) + 1))
        conn.executemany(
            f'INSERT INTO "{name}" VALUES ({marks})',
            ([r[t], *(float(r[i]) if r[i] != "" else None
         for i in wanted)] for r in reader),
        )
    # the time text ('2026-03-16 00:40:00+00:00') becomes
         epoch seconds, like samples.ts
    conn.execute(f'UPDATE "{name}" SET ts =
         unixepoch(ts)')
    return name


def series_sql(store: str, store_class: str, ts_id: str,
         key: str) -> tuple[str, dict]:
    """`key` makes the bound parameter name unique when
         two series go into one query."""
```

```
    if store_class == "Database":  # ref:Database; the
         model says backend sqlite, path in an env var
        if store not in sqlite_stores:
            sqlite_stores[store] = attach_sqlite(store)
        schema = sqlite_stores[store]
        sql = f"SELECT ts, value FROM {schema}.samples
         WHERE timeseries_id = :id_{key}"
        return sql, {f"id_{key}": ts_id}
    if store_class == "CsvFile":  # bts:CsvFile; the id is
         the column name
        table = csv_tables[store]
        col = quote_ident(ts_id, csv_columns[store])
        return f'SELECT ts, {col} AS value FROM temp."
         {table}"', {}
    raise ValueError(f"no reader for store class
         {store_class!r}")


# CSV stores are loaded once, with just the columns the
         references name.
csv_columns = {}
for r in refs[refs["store_class"] ==
         "CsvFile"].itertuples():
    csv_columns.setdefault(r.store_iri,
         set()).add(r.timeseries_id)
for store, columns in csv_columns.items():
    csv_tables[store] = load_csv(store, columns)

# 5. Resolve every reference of every FC1 point, and
         compare the two stores point by point.
COMPARE_SQL = """
WITH a AS ({sql_a}), b AS ({sql_b})
SELECT :point_id AS point_id,
       (SELECT count(*) FROM a) AS rows_sqlite,
       (SELECT count(*) FROM b) AS rows_csv,
       sum(a.value = b.value)   AS rows_equal,
       max(abs(a.value - b.value)) AS max_abs_diff
FROM a JOIN b ON b.ts = a.ts
"""
parts = []
for point_id, grp in refs.groupby("point_id"):
    by_class = {r.store_class: r for r in
         grp.itertuples()}
    db, cf = by_class["Database"], by_class["CsvFile"]
    sql_a, params_a = series_sql(db.store_iri, "Database",
         db.timeseries_id, "a")
    sql_b, params_b = series_sql(cf.store_iri, "CsvFile",
         cf.timeseries_id, "b")
    params = {"point_id": point_id} | params_a | params_b
    parts.append(
```

```
        pd.read_sql_query(COMPARE_SQL.format(sql_a=sql_a,
         sql_b=sql_b), conn, params=params)
    )
series_compare = pd.concat(parts, ignore_index=True)

RESULTS = {
    "refs": refs[["point_id", "store", "store_class",
         "timeseries_id"]],
    "series_compare": series_compare,
}

print("References of FC1's input points (a second store
         was added to each point in memory):")
print(RESULTS["refs"].to_string(index=False))
print("\nThe same series, read from SQLite and from the
         CSV file:")
print(series_compare.to_string(index=False))
```

Day 25 — Model-driven FC1 through the app's API (week 5 challenge)

Goal: run Day 14's FC1 rule with no column names and no ids written in the code: SPARQL finds the inputs, the app's API (or SQL on its database) supplies the series. Then run the same rule against the CSV storage from Day 24.

Read

Day 14 hard-coded `supply_fan_status`, `da_p_inwc` and `da_p_setpoint_inwc`. This week replaced each hard-coded fact with a lookup: roles (Day 21), tags in RDF (Day 22), stable ids (Day 23), references to stores (Day 24). Today they all work together.

`lab.Api` runs this repo's FastAPI app (`bldgts serve`) inside the Python process with FastAPI's `TestClient`: requests go straight into the app, no server and no network. It reads the model `model/building_50.ttl` and the database `var/timeseries.sqlite`. It offers two things we use:

- `api.sparql(query)` runs SPARQL against the model and returns a DataFrame. The query is Day 22's: roles and `equipRef`, starting from points that have a timeseries reference (pattern order, as before).
- `api.timeseries(point_id)` is `GET /api/points/{id}/timeseries`: the API looks up the point's reference, resolves it (the Day 24 job, done server-side) and returns `ts, value`.

Day 14's rule is unchanged. FC1 is the same four steps (gate and raw fault, streak id, row in streak, confirm) with the same tolerance (25 Pa) and confirmation (2 rows). The only new work is **getting the data in the shape the rule wants**, long → wide (Day 23) and inches of water → pascals (Day 6):

Step	`day25_pandas.py`	`day25_sql.py`
find the inputs	`api.sparql(FC1_POINTS_SPARQL)`	the same query, via `lab.Api`
get the series	`api.timeseries(point_id)` ×6, one `GET` each	none: SQLite reads `samples` directly
use the ids	the API is told the point; it returns the UUID it served (the script prints it equals the model's)	UUIDs go into a `TEMP` table `fc1_input`, joined to `samples`
long → wide	`long.pivot(index=["equipment_id","ts"], columns="role")`	`MAX(CASE WHEN i.role = :role_dp THEN s.value END)`
SI + FC1	numpy/pandas arithmetic, `groupby().shift()/cumsum()/cumcount()`	`CASE`, `LAG`, `SUM() OVER`, `ROW_NUMBER() OVER`

Both routes have the same cost shape: O(n) rows moved and one sort for the windows. Through HTTP each series is also serialized to JSON and back, so the API route is slower per row than reading SQLite directly. You pay for the indirection to get a stable interface: the client never sees the database, the table or the UUIDs it didn't ask for. The SQL script shows the other side: when you do own the database, the model's answer ("these UUIDs") feeds a *parameterized* query, and no id is written in the SQL.

Where do the role names ("fan-status", …) still appear? In the SPARQL filter and the SQL's `:role_*` parameters. They are the rule's **input contract**: that is all the rule knows about the building. Everything else, which point, which column, which store, which id, comes from the model.

Run

Needs `var/timeseries.sqlite` and the model (`uv run bldgts model build && uv run bldgts bootstrap`).

```
uv run python lessons/course.py run 25
```

That runs `day25_pandas.py`, then `day25_sql.py`. Both end with:

```
FC1 per AHU (hours, episodes). Day 14 printed the same
numbers from hard-coded columns:
equipment_id  raw_fault_h  confirmed_fault_h  episodes
       AHU_1       120.00             114.25        53
       AHU_2         3.42               1.50         5
```

These are Day 14's numbers (AHU_1 114.25 h in 53 episodes, AHU_2 1.50 h), from a script that names no column. The per-row fault flags are compared too, so the two scripts agree on every one of the 71,072 rows.

Try it

1. Add the **duct static too high** rule (Day 14, Try it 2) to the pandas script. Which part of the pipeline changes, and does it need any new input from the model?
2. Run the script for AHU_2's `discharge-air-temp` and `discharge-air-temp-sp` instead of the static pressure roles. What do you change? (Mind the units: they are °F in the database.)

Answers

1. Only the comparison (`>` and `+ TOL_PA`). It uses the same three roles, so no new input.
2. The two role strings in the SPARQL `FILTER` and the roles in `wide/h`; and the conversion: temperatures use `(F - 32) * 5 / 9`, not `* 249.08891`. The tolerance would be a temperature *difference*, which has no 32 in it (Day 6).

Week 5 challenge: the same rule, a different store

Day 24 gave every FC1 input point a second reference, to the AHU's CSV. Run FC1 **again, with the series read from the CSV storage**, using the model to find the points and Day 24's resolver to read them. You should get AHU_1 114.25 h in 53 episodes and AHU_2 1.50 h again. Try it before opening the answer.

Answer

Start from a copy of `day24_pandas.py`: it already builds the graph with the CSV references and defines `read_series` and the `refs` table (one row per point per store). Append this. It changes exactly one thing, which store the resolver reads (`WANT`); the rule itself is Day 14's:

```
PA_PER_INH2O, TOL_PA, CONFIRM_ROWS = 249.08891, 25.0, 2
WANT = "CsvFile"  # <- the only change from the SQLite
         run: which store's reference to follow

ROLE_QUERY = (
    PREFIXES
    + """
SELECT DISTINCT ?equip ?role ?point WHERE {          #
         DISTINCT: each point now has two references
  ?point ref:hasExternalReference ?r .
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
         "duct-static-pressure-sp"))
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ; bts:tagValue ?equip
         .
}"""
)
frames = []
for r in g.query(ROLE_QUERY):
    ref = refs[(refs.point_iri == str(r.point)) &
         (refs.store_class == WANT)].iloc[0]
    series = read_series(ref.store_iri, ref.store_class,
         ref.timeseries_id)  # the resolver
    frames.append(series.assign(equipment_id=local(r.equip),
         role=str(r.role)))
long = pd.concat(frames, ignore_index=True)
wide = long.pivot(index=["equipment_id", "ts"],
         columns="role", values="value").reset_index()
h = pd.DataFrame(
    {
        "equipment_id": wide.equipment_id,
        "ts": wide.ts,
        "fan_status": wide["fan-status"],
        "duct_static_pa": wide["duct-static-pressure"] *
         PA_PER_INH2O,
        "duct_static_sp_pa": wide["duct-static-pressure-
         sp"] * PA_PER_INH2O,
    }
).sort_values(["equipment_id", "ts"], ignore_index=True)
# ... then Day 25's steps 4-5 unchanged: raw fault,
         streaks, confirm, hours and episodes.
```

The CSV series are in the same units and on the same timestamps as the database's, so the rule gives the same answer: AHU_1 114.25 h, 53 episodes; AHU_2 1.50 h. Without the `DISTINCT` the pivot would raise "Index contains duplicate

entries", because each point would appear once per reference. For the SQL route, load the CSV into a `TEMP` table as `day24_sql.py` does and join it instead of `samples`.

The lesson: the rule, the SPARQL and the pivot did not change. Only the store the resolver was told to prefer did. That is what the model buys you.

Next: **Unit 4** goes deeper into graphs and SPARQL itself, then the BACnet scans that feed models like this one.

day25_pandas.py

```
"""Day 25, pandas: FC1 driven by the model, through the app's API. No column names or ids in the code.

    uv run python lessons/unit3-modeling/day25_pandas.py

Needs var/timeseries.sqlite and model/building_50.ttl (`uv run bldgts model build && uv run bldgts bootstrap`). lab.Api runs the repo's FastAPI app in-process (no server, no network). SPARQL finds FC1's input points by tag; each point's series comes from GET /api/points/{id}/timeseries; then Day 14's FC1 logic runs in SI. Compare the printout with day25_sql.py (and with Day 14: AHU_1 114.25 h in 53 episodes, AHU_2 1.50 h)."""

import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_PA = 25.0  # how far below setpoint counts as a fault (~0.1 inH2O)
CONFIRM_ROWS = 2  # 2 rows x 5 min = 10 minutes before a fault counts

# 1. Ask the model for FC1's inputs. Start from the points that have a timeseries reference: rdflib
#    evaluates patterns roughly in order, so a selective first pattern keeps every join small.
FC1_POINTS_SPARQL = """
PREFIX bts: <https://example.org/openfdd/bldgts#>
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?ahu ?role ?point ?timeseriesId WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseriesId .
  ?point bts:hasHaystackTag ?roleTag .
  ?roleTag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure", "duct-static-pressure-sp"))
  ?point bts:hasHaystackTag ?direct .
  ?direct bts:tagName "equipRef" ; bts:tagValue ?ahu .
}
ORDER BY ?ahu ?role
```

```
"""
with lab.Api() as api:
    found = api.sparql(FC1_POINTS_SPARQL)
    found["equipment_id"] = found["ahu"].str.rsplit("#",
         n=1).str[-1]  # IRI local name: AHU_1
    found["point_id"] = found["point"].str.rsplit("#",
         n=1).str[-1]
    fc1_inputs = found[["equipment_id", "role",
         "point_id", "timeseriesId"]].rename(
        columns={"timeseriesId": "timeseries_id"}
    )

    # 2. One GET per series. The API reports the UUID it
         served, which should be the model's UUID.
    series = {}
    for r in found.itertuples():
        s = api.timeseries(r.point_id)
        series[(r.equipment_id, r.role)] =
         s.assign(equipment_id=r.equipment_id,
         role=r.role)
        same_id = s.attrs["timeseries_id"] ==
         r.timeseriesId  # shown in the printout
        print(f"GET {r.point_id:<16} -> {len(s):>6}
         samples, API id == model id: {same_id}")

# 3. Long -> wide (Day 23), then SI (Day 6/14): inH2O ->
         Pa. Fan status is already 0/1.
long = pd.concat(series.values(), ignore_index=True)
wide = long.pivot(index=["equipment_id", "ts"],
         columns="role", values="value").reset_index()
h = pd.DataFrame(
    {
        "equipment_id": wide["equipment_id"],
        "ts": wide["ts"],
        "fan_status": wide["fan-status"],
        "duct_static_pa": lab.inh2o_to_pa(wide["duct-
         static-pressure"]),
        "duct_static_sp_pa": lab.inh2o_to_pa(wide["duct-
         static-pressure-sp"]),
    }
).sort_values(["equipment_id", "ts"], ignore_index=True)

# 4. Day 14's FC1: gate + raw fault, streak id, row within
         the streak, confirm
eq = h["equipment_id"]
raw = ((h["fan_status"] == 1) & (h["duct_static_pa"] <
         h["duct_static_sp_pa"] - TOL_PA)).astype(int)
new_streak = raw.ne(raw.groupby(eq).shift()).astype(int)
streak_id = new_streak.groupby(eq).cumsum()
streak_len = raw.groupby([eq, streak_id]).cumcount() + 1
```

```
fault = ((raw == 1) & (streak_len >=
        CONFIRM_ROWS)).astype(int)
fc1 = h[["equipment_id", "ts"]].assign(raw_fault=raw,
        streak_id=streak_id, fault=fault)

# 5. Aggregate: rows x 5 minutes = hours, and confirmed
        episodes
hours = fc1.groupby("equipment_id")[["raw_fault",
        "fault"]].sum() * 5 / 60
episodes = fc1[fc1["fault"] == 1].groupby("equipment_id")
        ["streak_id"].nunique()
summary = (
    hours.rename(columns={"raw_fault": "raw_fault_h",
        "fault": "confirmed_fault_h"})
    .assign(episodes=episodes)
    .fillna({"episodes": 0})
    .reset_index()
)

RESULTS = {"fc1_inputs": fc1_inputs, "fc1_per_row": fc1,
        "fc1_summary": summary}

print("\nFC1's inputs, found by the model:")
print(fc1_inputs.to_string(index=False))
print(f"\n{len(h):,} rows of input after the pivot.")
print("\nFC1 per AHU (hours, episodes). Day 14 printed the
        same numbers from hard-coded columns:")
print(summary.round(2).to_string(index=False))
```

day25_sql.py

```
"""Day 25, SQL: FC1 driven by the model. The UUIDs come
        from SPARQL; SQLite runs the rule on
`samples`.

    uv run python lessons/unit3-modeling/day25_sql.py

Needs var/timeseries.sqlite and model/building_50.ttl (`uv
        run bldgts model build && uv run bldgts
bootstrap`). lab.Api (the repo's FastAPI app, in-process)
        answers the SPARQL question "which
points are FC1's inputs?". The UUIDs it returns go into a
        TEMP table, and one parameterized query
pivots `samples`, converts to SI and runs Day 14's FC1. No
        column names or ids are written in this
script. Compare the printout with day25_pandas.py (and
        with Day 14: AHU_1 114.25 h in 53 episodes,
AHU_2 1.50 h)."""

import sqlite3
import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

# 1. Ask the model for FC1's inputs (same query as
        day25_pandas.py).
FC1_POINTS_SPARQL = """
PREFIX bts: <https://example.org/openfdd/bldgts#>
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?ahu ?role ?point ?timeseriesId WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseriesId .
  ?point bts:hasHaystackTag ?roleTag .
  ?roleTag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure",
        "duct-static-pressure-sp"))
  ?point bts:hasHaystackTag ?direct .
  ?direct bts:tagName "equipRef" ; bts:tagValue ?ahu .
}
ORDER BY ?ahu ?role
"""
with lab.Api() as api:
    found = api.sparql(FC1_POINTS_SPARQL)
```

```
found["equipment_id"] = found["ahu"].str.rsplit("#",
        n=1).str[-1]  # IRI local name: AHU_1
found["point_id"] = found["point"].str.rsplit("#",
        n=1).str[-1]
fc1_inputs = found[["equipment_id", "role", "point_id",
        "timeseriesId"]].rename(
    columns={"timeseriesId": "timeseries_id"}
)

# The API's database, read-only. The UUIDs become a TEMP
        table the query joins to `samples`.
conn = sqlite3.connect(f"file:{lab.db_path().as_posix()}?
        mode=ro", uri=True)
conn.execute(
    "CREATE TEMP TABLE fc1_input (equipment_id TEXT, role
        TEXT, timeseries_id TEXT PRIMARY KEY)"
)
conn.executemany(
    "INSERT INTO fc1_input VALUES (?, ?, ?)",
    list(fc1_inputs[["equipment_id", "role",
        "timeseries_id"]].itertuples(index=False)),
)

PARAMS = {
    "role_fan": "fan-status",  # the rule's three inputs,
        by role: the only names the rule knows
    "role_dp": "duct-static-pressure",
    "role_sp": "duct-static-pressure-sp",
    "pa_per_inh2o": lab.PA_PER_INH2O,  # the historian
        logged inches of water; fault math is SI
    "tol": 25.0,  # how far below setpoint counts as a
        fault (~0.1 inH2O)
    "confirm": 2,  # 2 rows x 5 min = 10 minutes before a
        fault counts
}

# 2.-4. One query, one CTE per step: pivot long -> wide,
        SI, then Day 14's FC1.
FC1_SQL = """
CREATE TEMP TABLE fc1 AS
WITH wide AS (                                  -- long ->
        wide: one row per (equipment, ts)
  SELECT i.equipment_id, s.ts,
         MAX(CASE WHEN i.role = :role_fan THEN s.value
         END) AS fan_status,
         MAX(CASE WHEN i.role = :role_dp  THEN s.value
         END) AS da_p_inwc,
         MAX(CASE WHEN i.role = :role_sp  THEN s.value
         END) AS da_p_sp_inwc
  FROM samples s
  JOIN fc1_input i ON i.timeseries_id = s.timeseries_id
```

```
  GROUP BY i.equipment_id, s.ts
), si AS (                                  -- SI units
  SELECT equipment_id, ts, fan_status,
         da_p_inwc * :pa_per_inh2o    AS duct_static_pa,
         da_p_sp_inwc * :pa_per_inh2o AS duct_static_sp_pa
  FROM wide
), base AS (                                -- gate +
         raw fault
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static_pa <
         duct_static_sp_pa - :tol
              THEN 1 ELSE 0 END AS raw_fault
  FROM si
), marks AS (                               -- a new
         streak starts where raw_fault changes
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER w
         THEN 0 ELSE 1 END AS new_streak
  FROM base
  WINDOW w AS (PARTITION BY equipment_id ORDER BY ts)
), streaks AS (                             -- running
         sum of starts numbers the streaks
  SELECT *, SUM(new_streak) OVER (PARTITION BY
         equipment_id ORDER BY ts) AS streak_id
  FROM marks
), ranked AS (                              -- row
         number inside each streak
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id,
         streak_id ORDER BY ts) AS streak_len
  FROM streaks
)
SELECT equipment_id, ts, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len >= :confirm
         THEN 1 ELSE 0 END AS fault
FROM ranked
"""
conn.execute(FC1_SQL, PARAMS)
conn.execute("CREATE INDEX temp.fc1_eq_ts ON fc1
         (equipment_id, ts)")

# 5. Rows x 5 minutes = hours, and confirmed episodes
SUMMARY_SQL = """
SELECT equipment_id,
       sum(raw_fault) * 5 / 60.0 AS raw_fault_h,
       sum(fault) * 5 / 60.0     AS confirmed_fault_h,
       count(DISTINCT CASE WHEN fault = 1 THEN streak_id
         END) AS episodes
FROM fc1
GROUP BY equipment_id
ORDER BY equipment_id
"""
```

```
fc1 = pd.read_sql_query("SELECT * FROM fc1 ORDER BY
        equipment_id, ts", conn)
summary = pd.read_sql_query(SUMMARY_SQL, conn)

RESULTS = {"fc1_inputs": fc1_inputs, "fc1_per_row": fc1,
        "fc1_summary": summary}

print("FC1's inputs, found by the model:")
print(fc1_inputs.to_string(index=False))
print(f"\n{len(fc1):,} rows of input after the pivot.")
print("\nFC1 per AHU (hours, episodes). Day 14 printed the
        same numbers from hard-coded columns:")
print(summary.round(2).to_string(index=False))
```

Reading — Week 5

The lessons first, then these for exact behavior.

- Brick: timeseries storage (ref schema)
- Project Haystack: defs
- Project Haystack: RDF
- RFC 9562: UUIDs (version 5 = name-based)
- rdflib documentation
- FastAPI testing (TestClient)